

Más UML

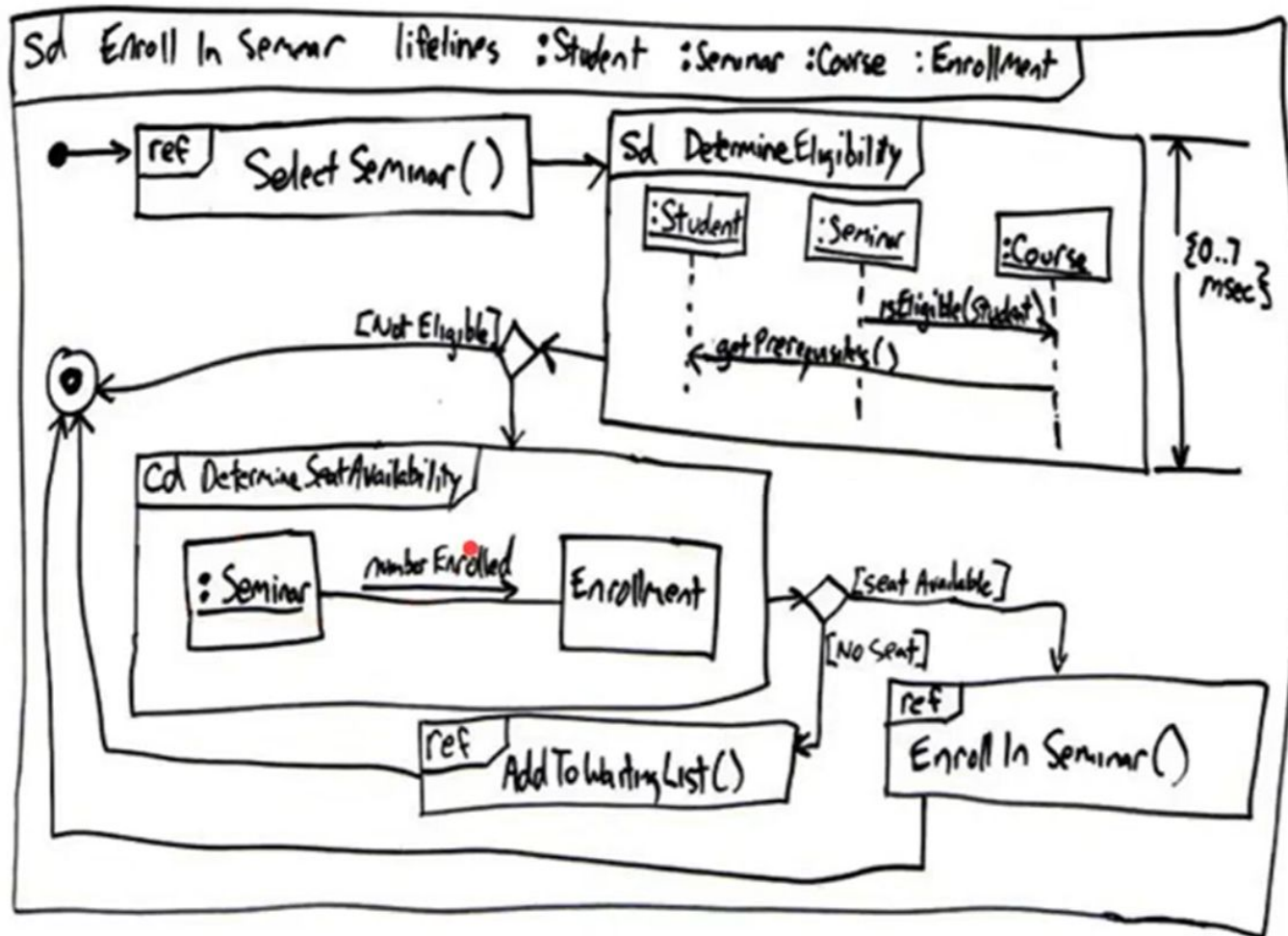


Diagrama de secuencia

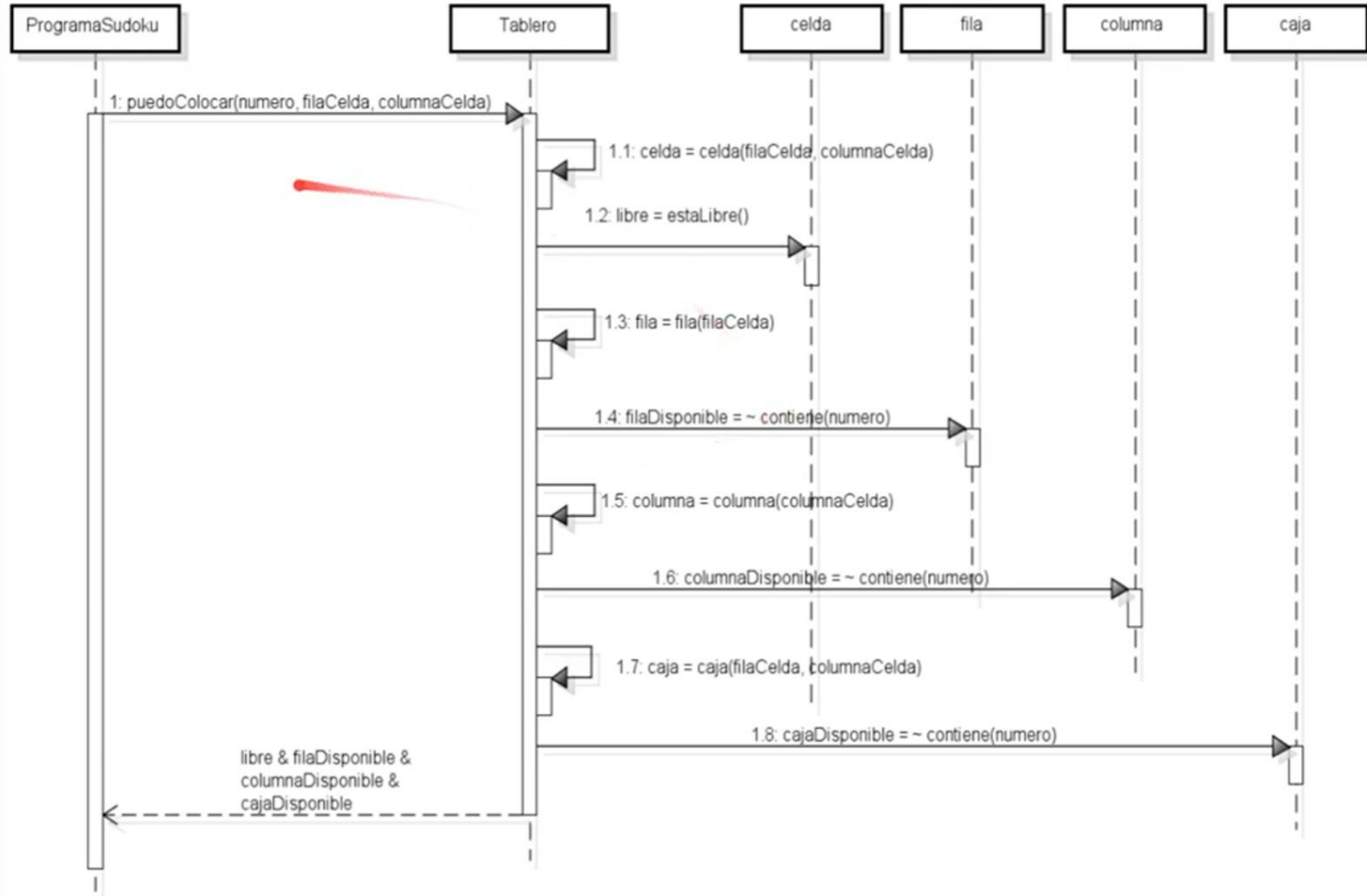
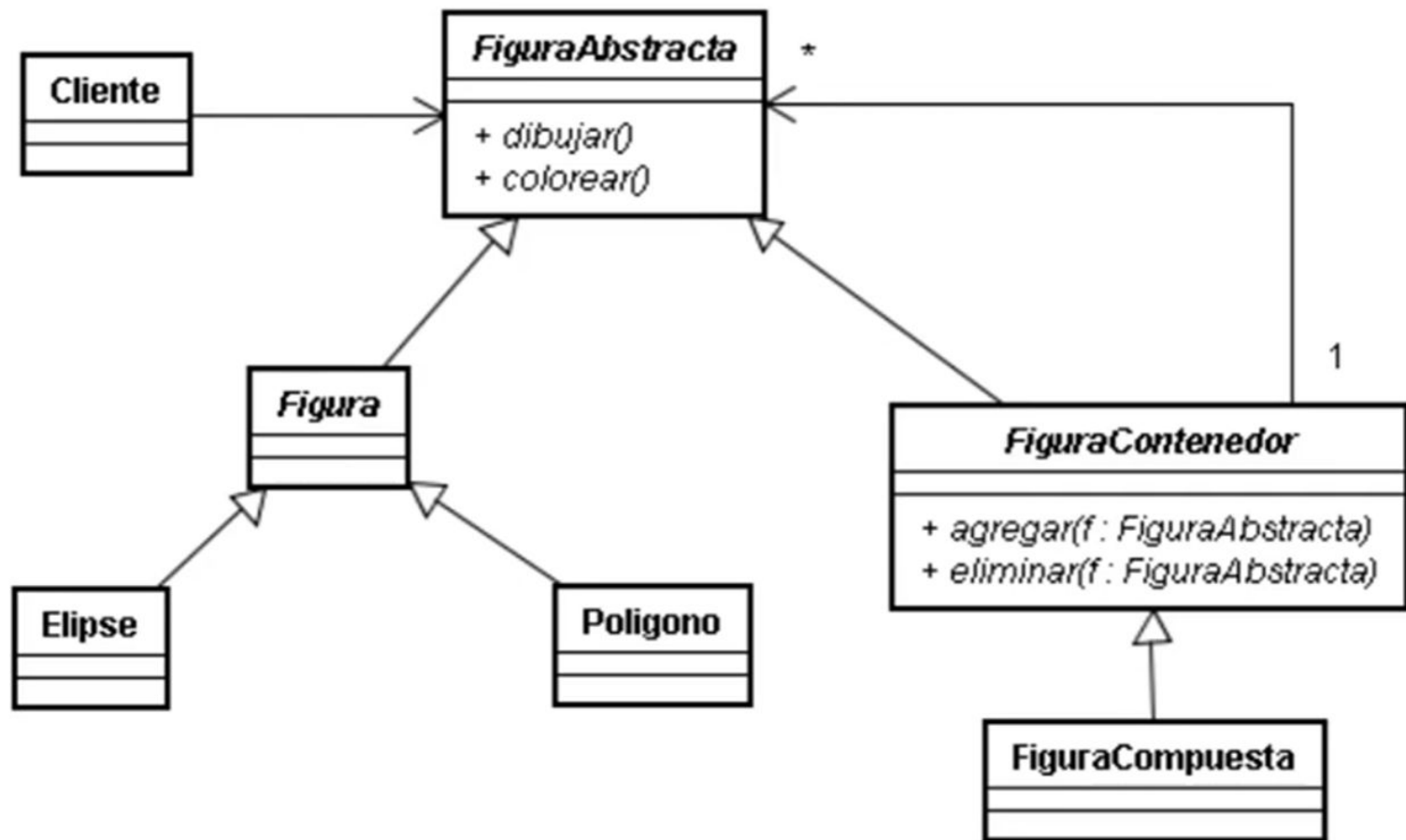


Diagrama de clases

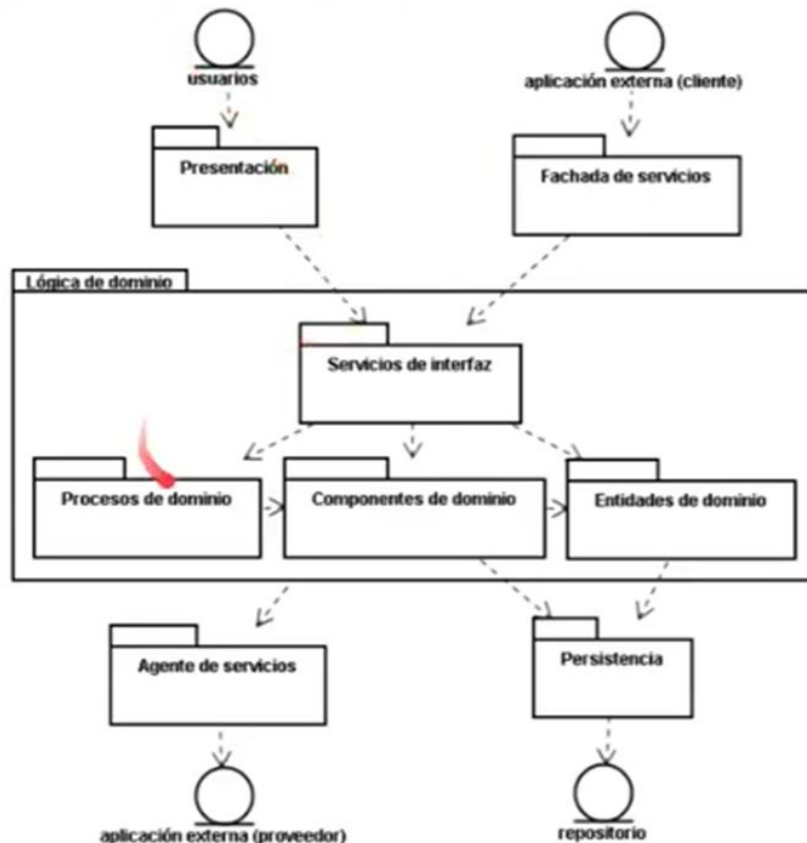


Paquetes

Agrupación de clases

En Java, anidables

Para manejar la complejidad y modularizar



Paquetes en Java

Toda clase está en un paquete

Hay un paquete “default”: mala idea...

ArrayList es java.util.ArrayList

```
import java.util.*;
```

```
import java.util.ArrayList;
```

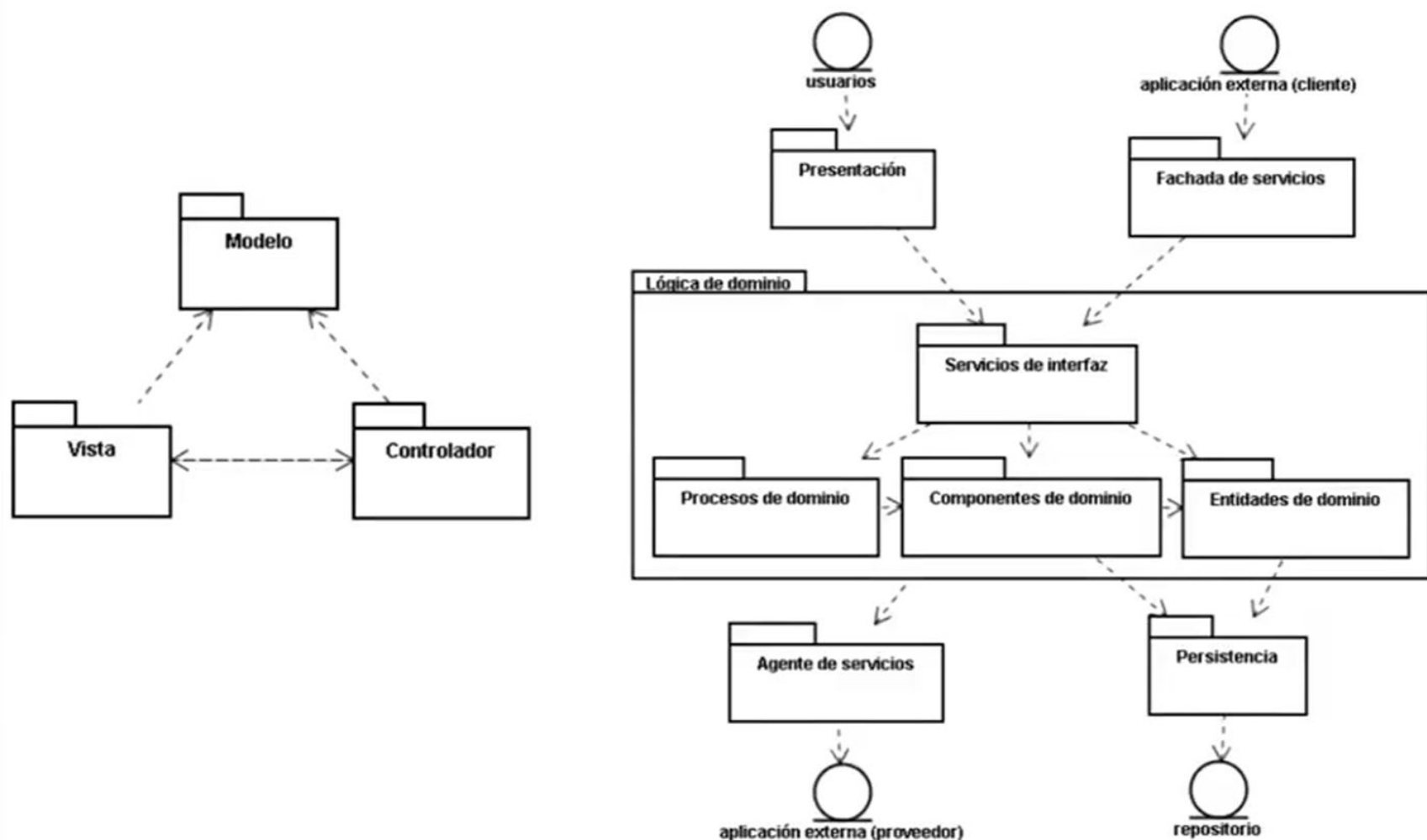
Sirve para resolución de nombres

Cada clase pública en un archivo fuente separado

El paquete se indica en una cláusula “package”

```
package carlosFontela.cuentas;
```

Diagramas de paquetes



Estados, eventos, transiciones

Estado

representado por el conjunto de valores adoptados por los atributos de un objeto en un momento dado
situación de un objeto durante la cual satisface una condición, realiza una actividad o espera un evento

Evento

Estímulo que puede disparar una transición de estados
Especificación de un acontecimiento significativo
Señal recibida, cambio de estado o paso de tiempo
Síncrono o asíncrono

Diagrama de estados UML: estados civiles (1)

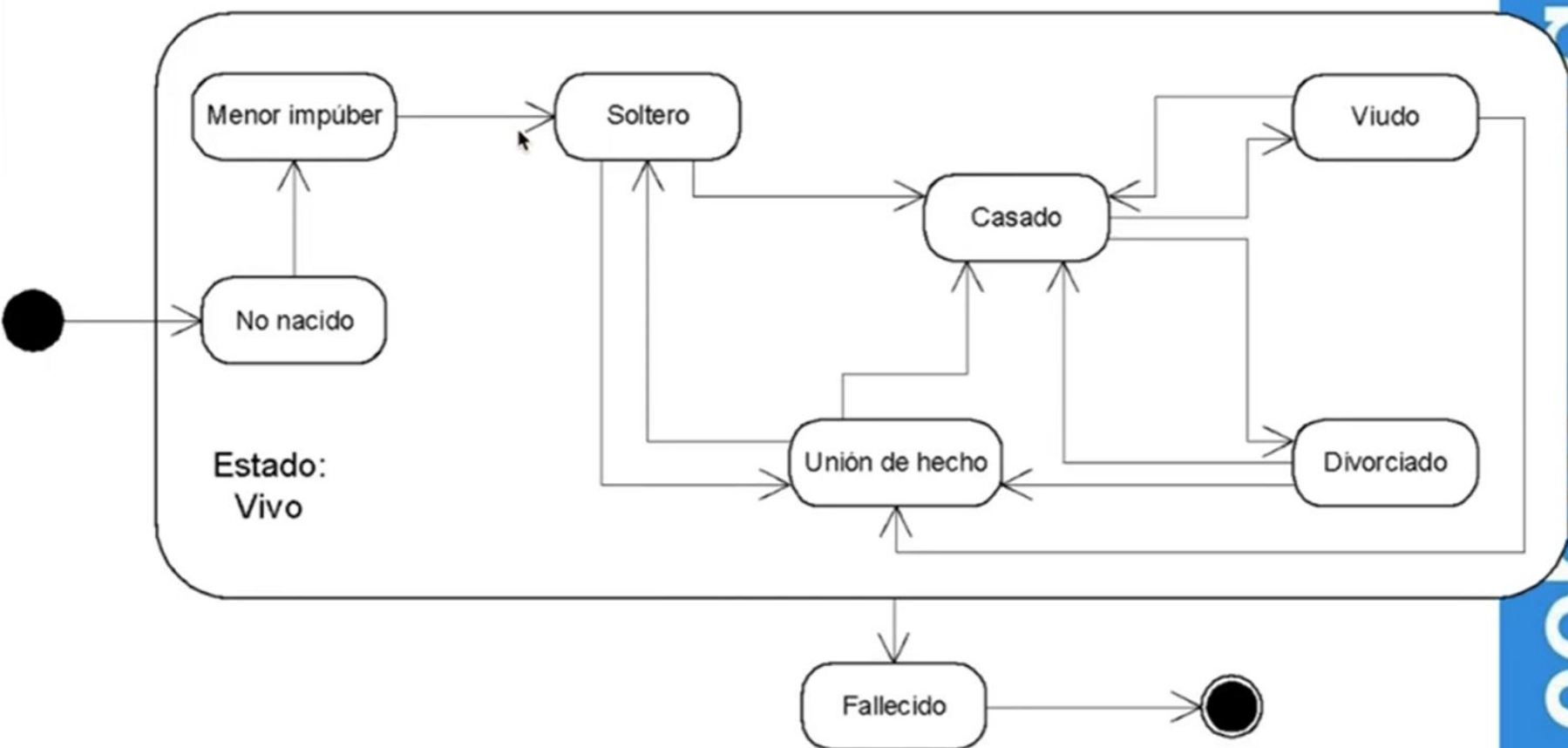


Diagrama de estados UML: estados civiles (2)

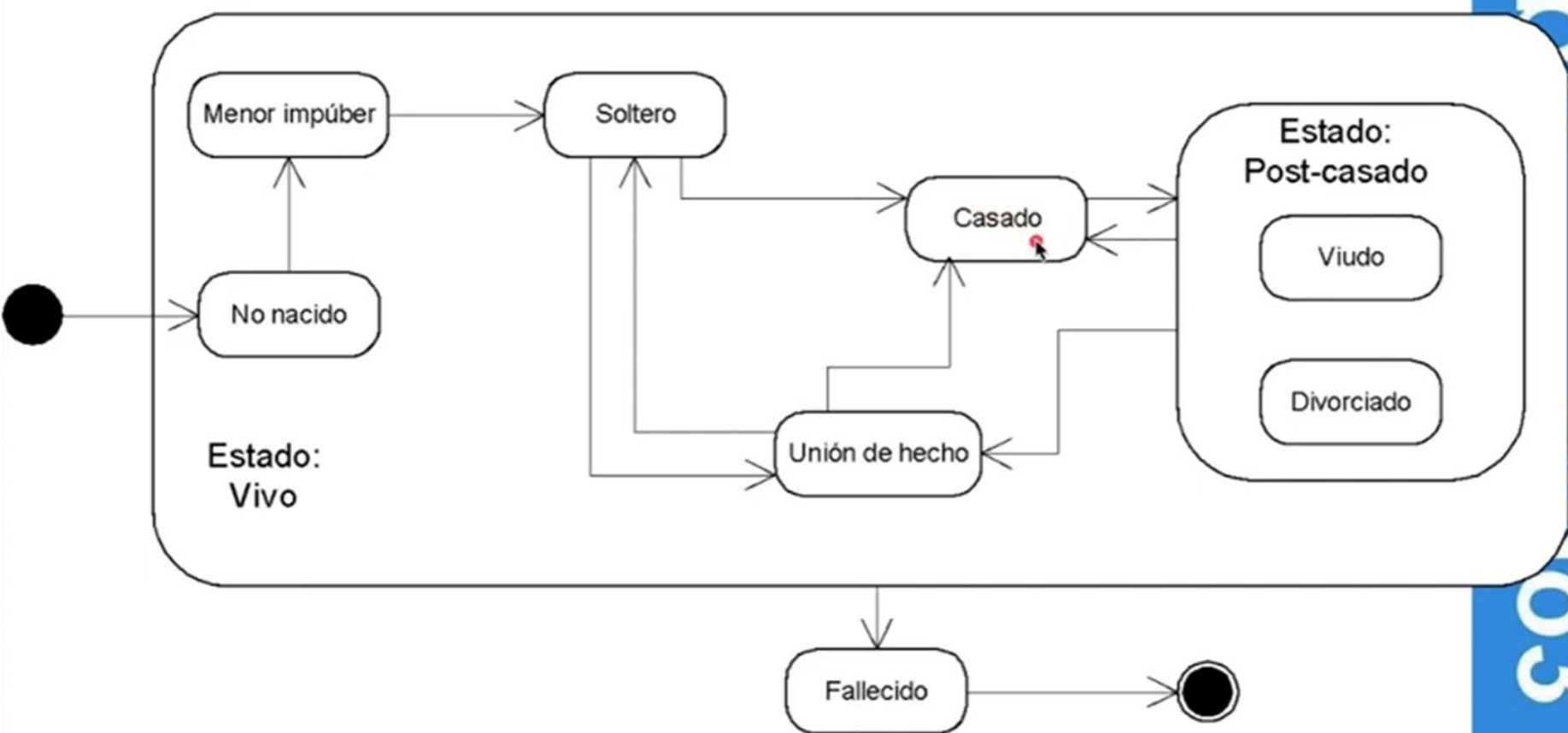
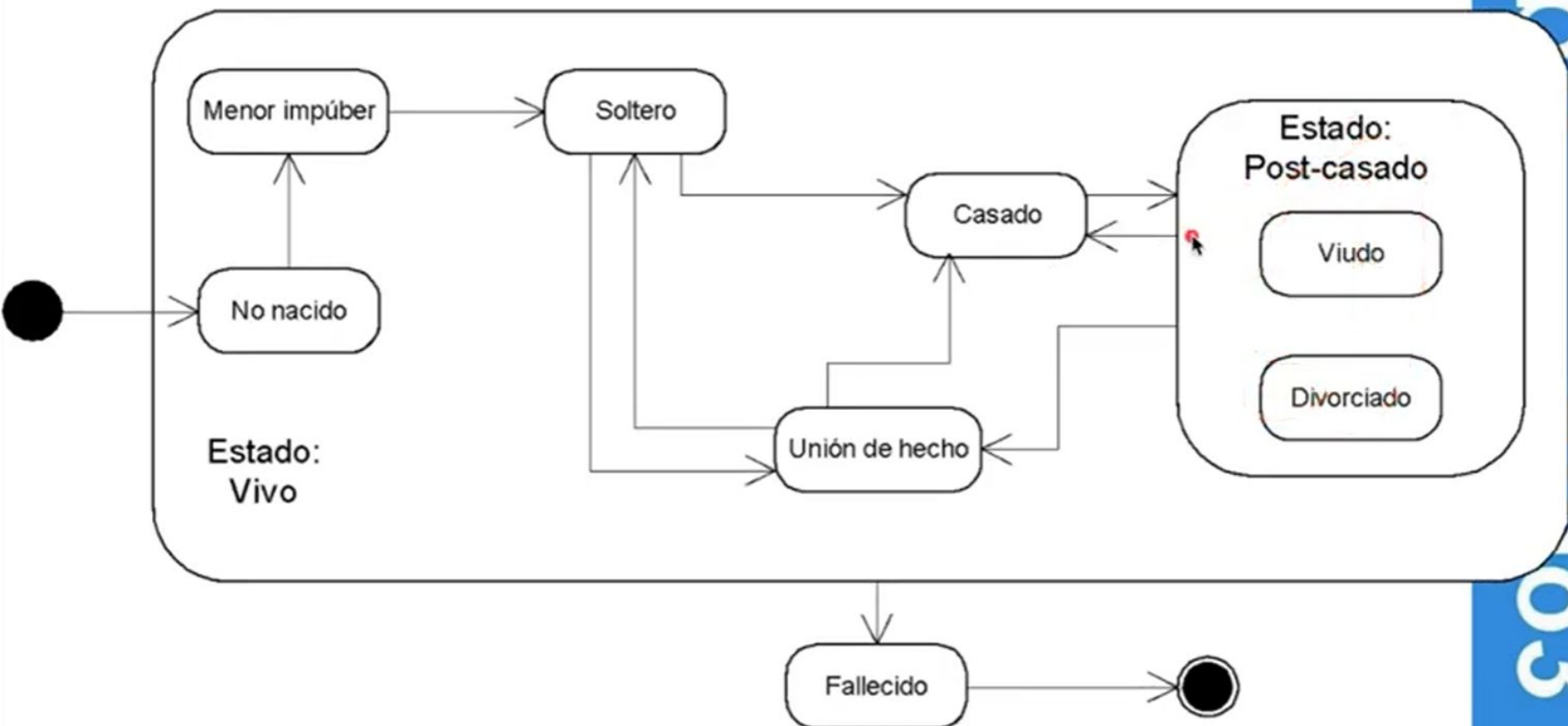


Diagrama de estados UML: estados civiles (2)



UML



Usos de UML

Para discutir diseños antes y durante la construcción

Para generar documentos que sirvan después de la construcción

Destinatarios humanos

Es lo más habitual

Generación de software

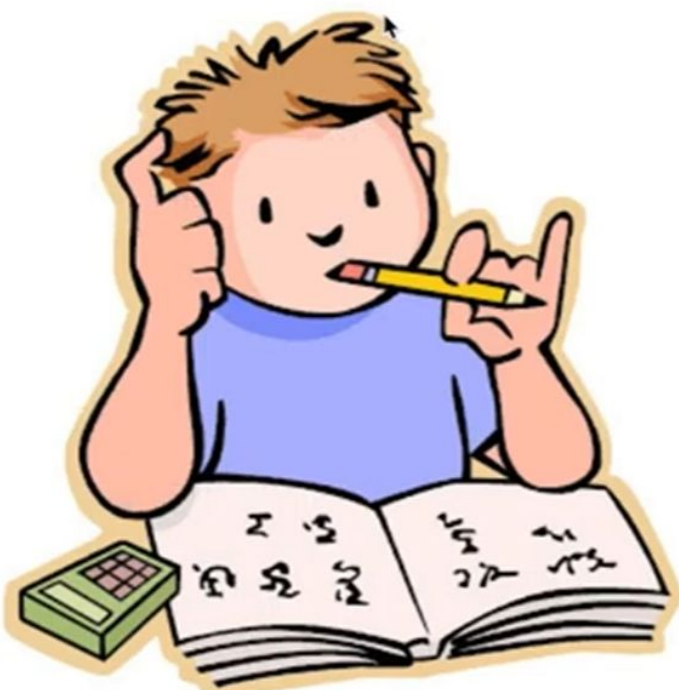
Model Driven Development

No es el foco en Algoritmos III

Recapitulación: preguntas

¿Cuándo usarían diagramas de estados?

¿Por qué enseñamos / exigimos UML en un curso de programación?



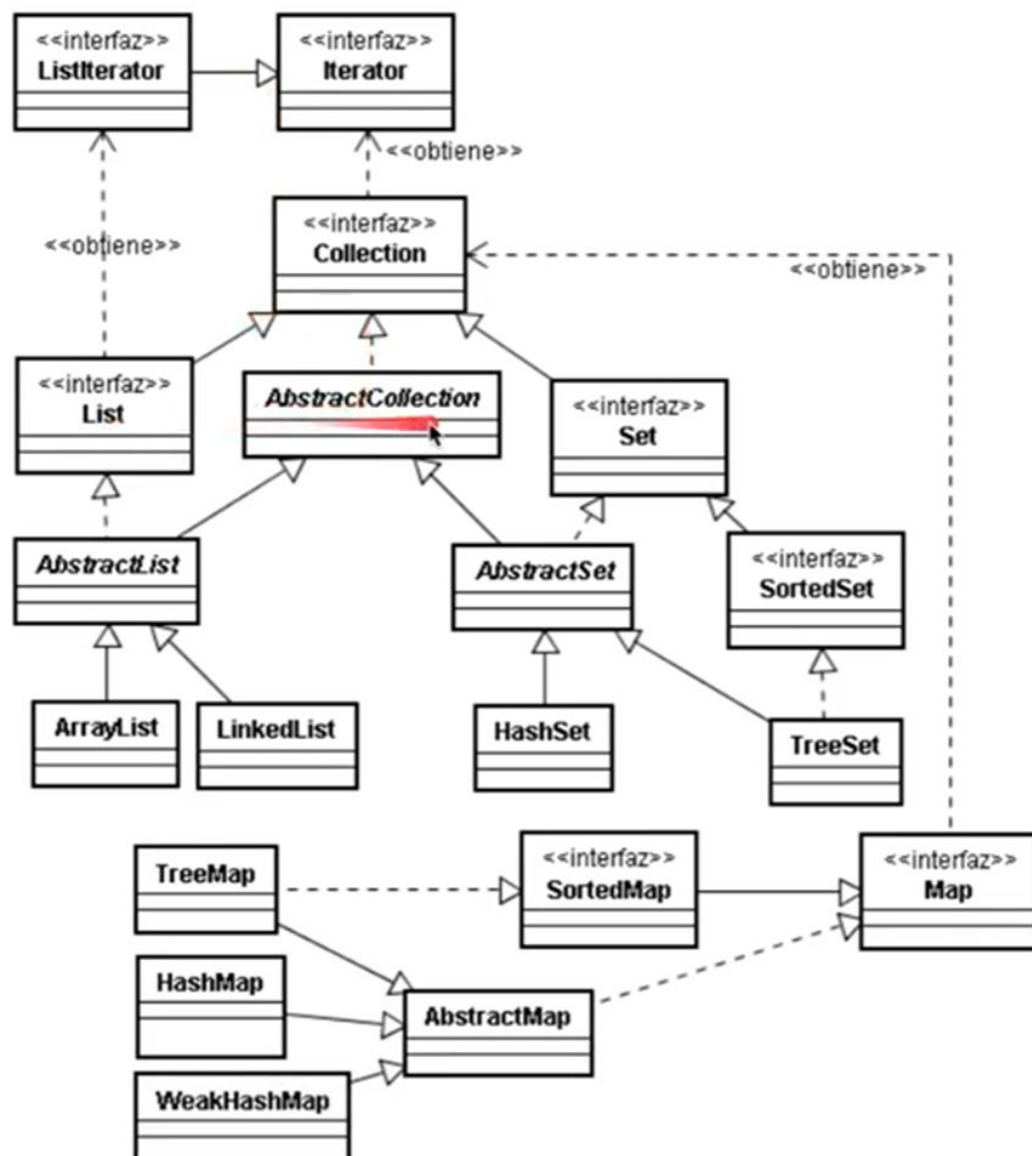
Colecciones, iteradores y genericidad (Java)

fiuba

algoritmo3



Colecciones de java.util 1.4



Java 1.4 $\approx$ Smalltalk (en las colecciones)

Elementos de tipo Object => sirven para cualquier tipo de datos

```
unaLista.add(new Cuenta());  
unaLista.add(new Elipse());
```

¡Pero no es Smalltalk!

No admiten tipos primitivos

```
unaLista.add(4); // no funciona en Java 1.4
```

“Cualquier tipo” al recuperar: ¿?

```
Cuenta c = unaLista.get(pos);  
    // error: Cuenta no es Object  
Cuenta c = (Cuenta) (unaLista.get(pos));
```

Iteradores: definición y uso

Objetos que saben cómo recorrer una colección, sin ser parte de ella

Interfaz:

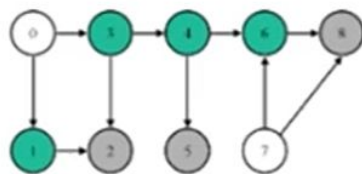
- Tomar el primer elemento

- Tomar el elemento siguiente.

- Chequear si se termina la colección

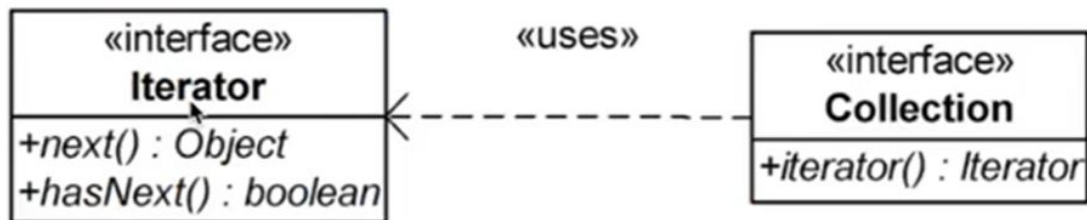
Un ejemplo:

```
List vector = new ArrayList();  
for(int j = 0; j < 10; j++) vector.add(j);  
Iterator i = vector.iterator();    // pido un iterador para vector  
while ( i.hasNext() )              // recorro la colección  
    System.out.println( i.next() );
```



Iteradores y colecciones

Toda clase que implemente Collection puede generar un Iterator con el método iterator



Nótese que Iterator es una interfaz

Pero está implementada para las colecciones definidas en `java.util`.

Iteradores: para qué

Llevan la abstracción a los recorridos de colecciones

Facilitan cambios de implementación

```
Collection lista = new ArrayList ( );
```

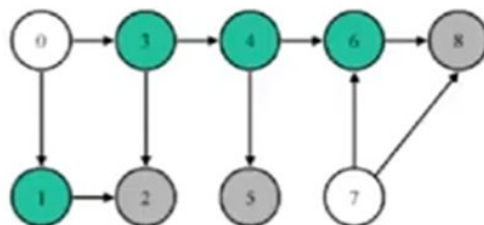
```
Iterator i = lista.iterator(); // pido un iterador para lista
```

```
while ( i.hasNext() ) // recorro la colección
```

```
    System.out.println( i.next() );
```

No se necesita trabajar con el número de elementos

Convierten a las colecciones en simples secuencias



Iteradores en Smalltalk

Modelo más simple

Analizar

```
celdas do: [:celda | (celda contiene: numero)
```

```
  ifTrue: [ encontrado := true ] ].
```

celdas referencia una instancia de

OrderedCollection

Ejercicio: lista circular (1)

¿Qué es una lista circular?

Definición: una lista que se recorre indefinidamente, de modo tal que al último elemento le sigue el primero

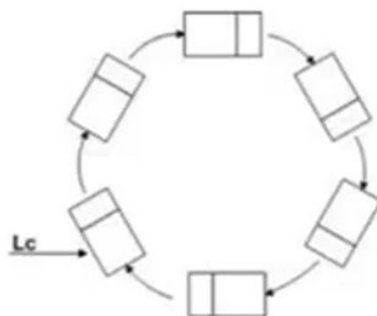
Es un caso particular de LinkedList

¿Qué cambia?

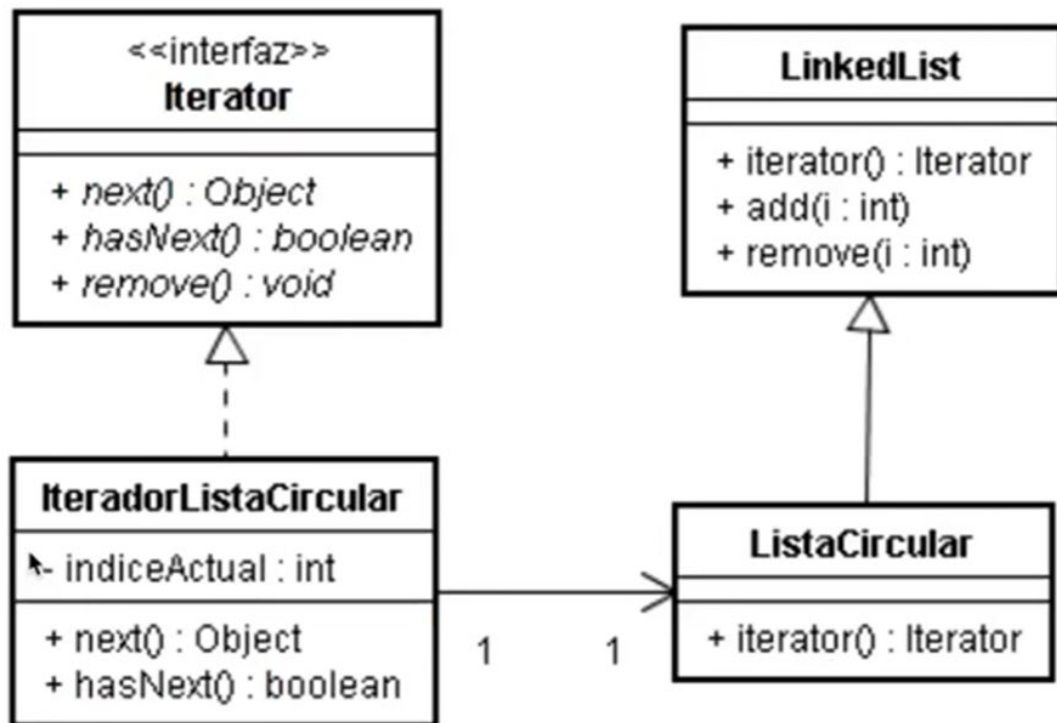
¿Nada?

¿Sólo la forma de recorrerla?

=> El iterador es diferente



Ejercicio: lista circular (2)

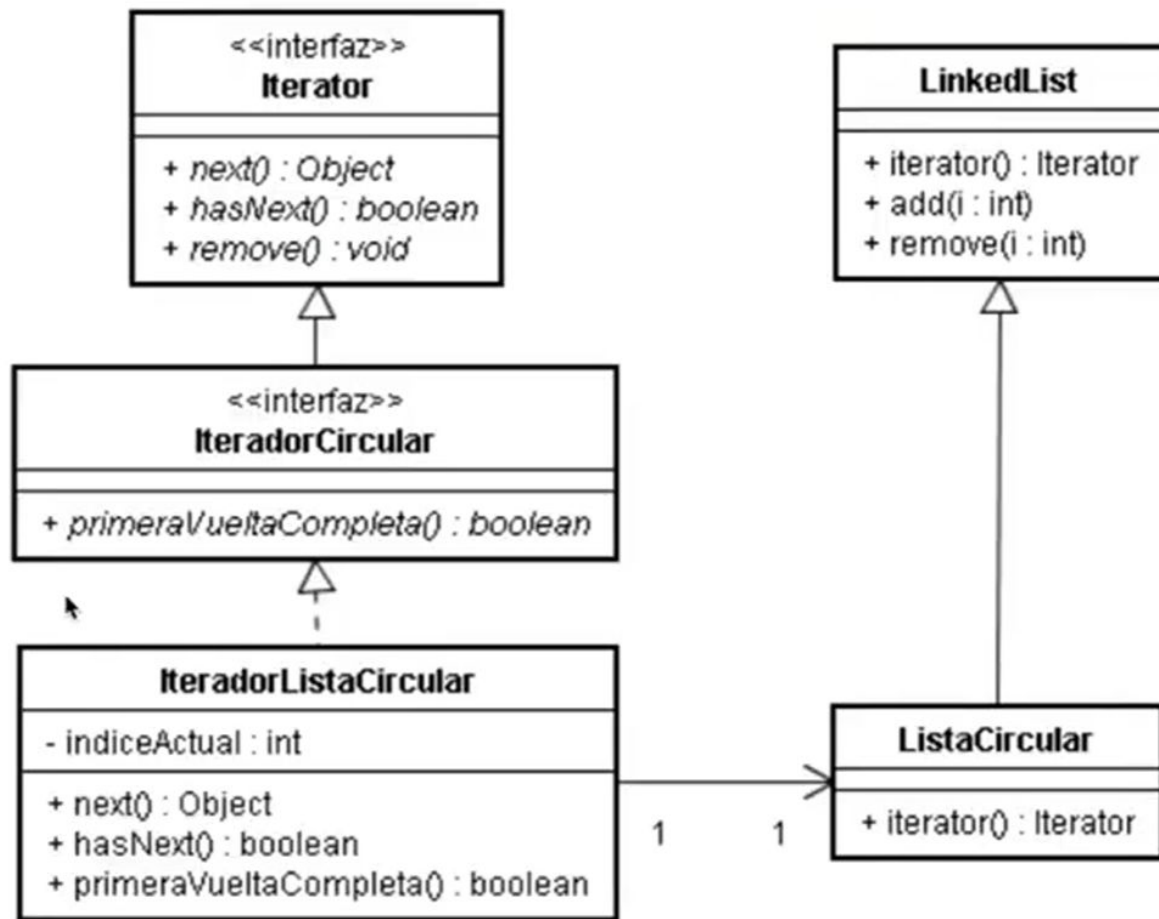


Ejercicio: lista circular (3)

```
public class ListaCircular extends LinkedList {  
    public Iterator iterator( ) {  
        return new IteradorListaCircular(this);  
    }  
}
```

Implementar la clase IteradorListaCircular
Con sus métodos next() y hasNext()

Ejercicio lista circular: otra visión



Genericidad: los tipos también son parámetros

Sin

```
List v = new ArrayList( );  
String s1 = "Una cadena";
```

```
// s1 pasa como Object:  
v.add(s1);
```

```
// get obtiene un Object  
// "puenteo" el chequeo:  
String s2 =  
    (String)v.get(0);
```

Con

```
List<String> v =  
    new ArrayList<String>( );  
String s1 = "Una cadena";
```

```
// el compilador verifica  
// que s1 sea un String:  
v.add(s1);
```

```
String s2 = v.get(0);
```


Genericidad (más allá)

En métodos, el compilador infiere el tipo genérico:

```
public static <T> void eliminarElemento (List<T> lista, int i) { ... }  
eliminarElemento (listaConcreta, 6);
```

Mejoras:

Robustez en tiempo de compilación

Legibilidad 

Cuestiones avanzadas

```
public static <T extends Comparable > void ordenar (T[ ] v) { ... }  
public static <T > copy (List<T> destino, List<? extends T> origen) { ... }  
public static <T, S extends T> copy (List<T> destino, List<S> origen) { ... }
```

Genericidad: Java vs. .NET

Java

usa la genericidad sólo
para tiempo de
compilación

No llega al bytecode =>
compatibilidad hacia
atrás

No hay información del
tipo completa en
tiempo de ejecución

.NET

mantiene la información
de tipos completa
hasta tiempo de
ejecución

Pero generó una biblioteca
de clases nueva => sin
compatibilidad hacia
atrás

Recapitulación: preguntas

¿Qué ventajas aporta la genericidad cuando trabajamos con colecciones?

¿Para qué se usan las interfaces?



Claves

- Inicialización debería dejar al objeto en un estado válido

- Excepciones cuando no hay suficiente información en el contexto del problema

- UML es una herramienta de modelado

 - Para discutir diseños antes del código

 - Para generar documentos que sirvan después de la construcción

- Genericidad lleva la idea del tipeo estático a las colecciones

- También permite que los tipos sean parámetros

Lecturas obligatorias

“What’s a Model For?”, Martin Fowler

<http://martinfowler.com/distributedComputing/purpose.pdf>



Lecturas optativas

UML Distilled 3rd Edition, Martin Fowler,
capítulo 1 “Introduction”

Hay edición castellana de la segunda edición
Debería estar en biblioteca

UML para programadores Java, Robert Martin,
capítulo 2 “Trabajar con diagramas”

No está en la Web ni en la biblioteca

Orientación a objetos, diseño y programación,
Carlos Fontela 2008, capítulo 9:
“Excepciones”

Qué sigue

Ejercicio avanzado de diseño

Repaso y revisión lecturas obligatorias

Calidad de código

Primer parcial TBD

